

บทที่ 10

พื้นฐานการเขียนโปรแกรม และการนำไปใช้

ผู้ช่วยศาสตราจารย์ ดร.ธิดาภัทร อนุชาญ



วัตถุประสงค์การเรียนรู้

- เพื่อให้นิสิตรู้จักชนิดโปรแกรมภาษาต่างๆ
- เพื่อให้นิสิตเข้าใจความแตกต่างของค่าคงที่และตัวแปร
- เพื่อให้นิสิตรู้จักชนิดข้อมูลต่างๆ ฟังก์ชัน, ตัวดำเนินการ และนิพจน์ในภาษาโปรแกรม
- เพื่อให้นิสิตสามารถเขียนและอธิบายคำสั่งพื้นฐานของโปรแกรมได้

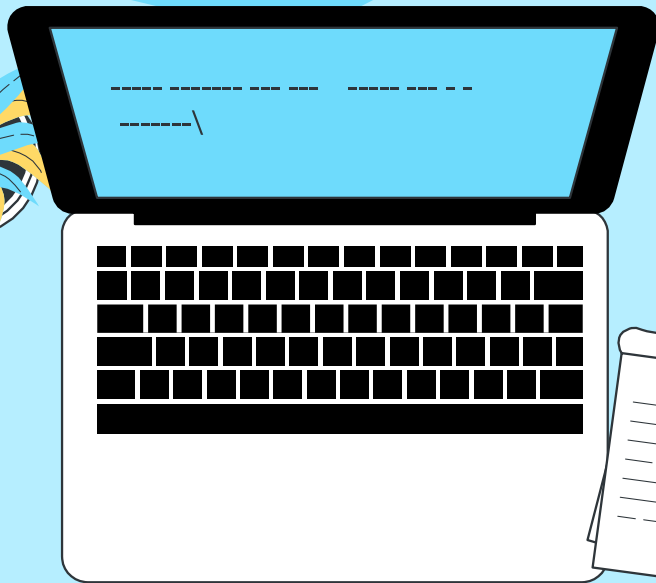
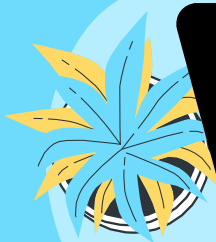


Table of contents

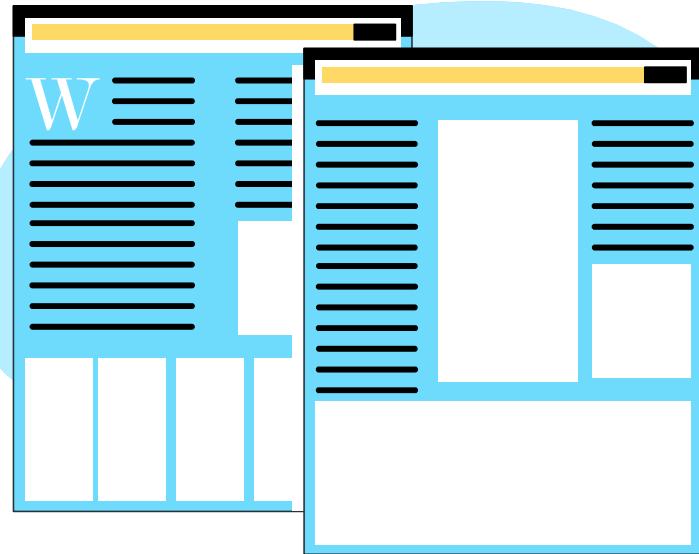
- 1 ภาษาสำหรับการพัฒนาโปรแกรม
- 2 โครงสร้างทางตรรกะในการเขียนโปรแกรม
- 3 ข้อมูล ตัวแปร ฟังก์ชัน และตัวดำเนินการ

01

ภาษาสำหรับการพัฒนาโปรแกรม



ภาษาที่ใช้สำหรับการพัฒนาโปรแกรมในระบบคอมพิวเตอร์ เรียกว่า **ภาษาคอมพิวเตอร์** หมายถึง สัญลักษณ์ อักขระบนคอมพิวเตอร์ที่ถูกกำหนดขึ้นมา เพื่อให้มนุษย์สามารถสั่งงาน และควบคุมให้คอมพิวเตอร์ทำงานได้ตรงกับความต้องการ ซึ่งภาษาคอมพิวเตอร์นั้นมีอยู่หลายชนิด หลายแบบที่นำมาพัฒนาโปรแกรม



ชนิดของภาษาคอมพิวเตอร์

ยุคที่ 1 : ภาษาเครื่อง (Machine Languages)

- ภาษาเครื่อง ถือเป็นภาษาโปรแกรมซึ่งใช้ในการพัฒนาคอมพิวเตอร์ **ระดับต่ำที่สุด** ในระยะแรกเริ่มคำสั่งที่ใช้เขียนประกอบด้วย **เลขฐานสองเท่านั้น**
- โปรแกรมเมอร์ต้องมีความรู้เกี่ยวกับการทำงานภายในของเครื่องคอมพิวเตอร์เป็นอย่างดี เพราะต้องระบุถึงตำแหน่งการเก็บข้อมูลในหน่วยความจำ
- คอมพิวเตอร์แต่ละเครื่องจะมีภาษาเครื่องแตกต่างกันขึ้นอยู่กับชนิดของเครื่องคอมพิวเตอร์และหน่วยประมวลผลกลาง (Central Processor Unit: CPU) โดยมีรูปแบบคำสั่งเฉพาะเครื่อง
- โปรแกรมเมอร์จึงไม่นิยมที่จะเขียนโปรแกรมด้วยภาษาเครื่อง เพราะทำการแก้ไขและเขียนโปรแกรมได้ยาก ทำให้เกิดความยุ่งยากในการจดจำ และเขียนคำสั่งต้องใช้เวลามากในการเขียนโปรแกรมรวมทั้งการหาข้อผิดพลาดจากการทำงานของโปรแกรม และโปรแกรมที่เขียนขึ้นทำงานเฉพาะคอมพิวเตอร์ที่มีฮาร์ดแวร์เดียวกันเท่านั้น (Machine Dependent)
- ข้อดีของภาษาเครื่อง คือ สามารถเขียนโปรแกรมควบคุมการทำงานของคอมพิวเตอร์ได้โดยตรง และสั่งงานให้คอมพิวเตอร์ทำงานได้อย่างรวดเร็ว

คำสั่งภาษาเครื่อง (Machine Code)

0010 0000

0100 0000

0011 0000

ความหมาย

โหลดข้อมูลจากหน่วยความจำ

ดำเนินการบวกข้อมูล

เก็บข้อมูลลงในหน่วยความจำ

ยุคที่ 2 :ภาษาแอสเซมบลี (Assembler Languages)

- เป็นภาษาที่อยู่ในลำดับขั้นที่สูงกว่าภาษาเครื่อง ซึ่งพัฒนาให้เขียนง่ายกว่าภาษาเครื่อง อาจเรียกว่า **ภาษาสัญลักษณ์ (Symbolic Language)** เพราะใช้สัญลักษณ์ แทน คำสั่งปฏิบัติงานและตำแหน่งที่เก็บข้อมูล
- ในภาษาแอสเซมบลี จะแทน**คำสั่งปฏิบัติงานต่าง ๆ ด้วยตัวอักษร** ซึ่งเรียกว่า mnemonics (memory aids) ดังนั้น โปรแกรมเมอร์ที่เขียนคำสั่งด้วยภาษาแอสเซมบลี **ไม่สามารถสื่อสารกับคอมพิวเตอร์ได้โดยตรง** ต้องมีล่ามในการแปลตัวอักษรต่าง ๆ เหล่านี้ให้เป็นภาษาเครื่องอีกที ตัวล่ามที่ว่่านี้นี้เรียกว่า **แอสเซมเบอร์ (assemblers)**

Mnemonic code	คำสั่งภาษาเครื่อง	ความหมาย
LOAD	0010 0000	โหลดข้อมูลจากหน่วยความจำ
ADD	0100 0000	ดำเนินการบวกข้อมูล
SUB	1101 0000	ดำเนินการลบข้อมูล
MOV	1001 0000	ย้ายข้อมูลเข้าออกจากหน่วยความจำ
STROE	0011 0000	เก็บข้อมูลไว้ในหน่วยความจำ

CALL MySub ;transfer of control

MOV AX, 5 ;data transfer

ADD AX, 20 ;arithmetic

JZ Next 1 ;logical (jump if zero)

IN A 1, 20 ;input/output (read from hardware port)

RET ; return

สรุปคำสั่งที่เขียนด้วยภาษาโปรแกรมคอมพิวเตอร์ ในยุคที่ 1 และที่ 2 จะต้องใช้
เทคนิคการเขียนโปรแกรมสูง เพราะมีความยืดหยุ่นในการเขียนน้อยมาก และมีความยากใน
การเขียนคำสั่งสำหรับผู้เขียนโปรแกรม แต่สามารถควบคุมและเข้าถึงการทำงานของเครื่อง
คอมพิวเตอร์ได้โดยตรง และมีความรวดเร็วกว่าการใช้ภาษาระดับอื่น ๆ

ยุคที่ 3 : ภาษาระดับสูง (High-level Language)

- มีโครงสร้างภาษา และชุดคำสั่งเหมือนกับภาษาอังกฤษ รวมทั้งสามารถใช้เทคนิคทางคณิตศาสตร์ในการคำนวณได้ด้วย ทำให้ผู้เขียนโปรแกรมสะดวกในการเขียนคำสั่ง และแสดงผลลัพธ์ได้ตามต้องการ ลดความยุ่งยากในการเขียนโปรแกรมลงได้มาก ทั้งยังทำให้เกิดการใช้งานคอมพิวเตอร์ เพื่อการประมวลผลเพิ่มขึ้น
- การเขียนโปรแกรม จะต้องใช้ตัวแปลภาษา ที่เรียกว่า คอมไพเลอร์ (Compiler) เพื่อแปลภาษาระดับสูงโดยการตรวจสอบไวยากรณ์ของภาษาระดับสูง ไปเป็นภาษาเครื่องเพื่อสั่งให้เครื่องคอมพิวเตอร์ทำงานต่อไป โดยคอมไพเลอร์ของภาษาระดับสูงแต่ละภาษาจะแปลเฉพาะภาษาของตนเอง และทำงานได้เฉพาะเครื่องคอมพิวเตอร์ชนิดเดียวกันเท่านั้น เช่น คอมไพเลอร์ของภาษา COBOL บนเครื่องไมโครคอมพิวเตอร์ จะแปลภาษาเฉพาะคำสั่งของภาษา COBOL และจะทำงานได้บนเครื่องคอมพิวเตอร์ที่เหมือนกันเท่านั้น ถ้าต้องการนำไปใช้กับเครื่องคอมพิวเตอร์แบบอื่นๆ เช่น เมนเฟรม จะต้องใช้คอมไพเลอร์ของภาษา COBOL แบบใหม่

ยุคที่ 3 : ภาษาระดับสูง (High-level Language)

- ตัวอย่างของภาษาคอมพิวเตอร์ระดับสูง ได้แก่ ภาษา BASIC, COBOL, FORTRAN และ C สามารถเขียนโปรแกรมแก้ปัญหาเฉพาะด้าน เช่น การควบคุมหุ่นยนต์ การสร้างภาพกราฟิก ได้เป็นอย่างดีเพราะมีความยืดหยุ่นและเหมาะกับการใช้งานทั่ว ๆ ไปได้
- สรุปภาษาโปรแกรมคอมพิวเตอร์ในยุคที่ 3 มีการเขียนโปรแกรมที่ง่ายกว่าในยุคที่ 2 สามารถทำงานได้บนเครื่องคอมพิวเตอร์หลายระดับ (Machine Independent) โดยต้องใช้ควบคู่กับตัวแปลภาษา (Compiler or Interpreter) สำหรับเครื่องนั้น ๆ และมีความยืดหยุ่นในการแก้ปัญหาได้มากกว่าภาษาระดับต่ำ

ยุคที่ 4 : ภาษาระดับสูงมาก (Very high-level Language)

- เป็นภาษาที่ใช้ในการเขียนโปรแกรมด้วยคำสั่งสั้น ๆ และง่ายกว่าภาษาในยุคก่อน ๆ
- มีการทำงานแบบไม่จำเป็นต้องบอกลำดับของขั้นตอนการทำงาน (Nonprocedural language) เพียงนักเขียนโปรแกรมกำหนดว่าต้องการให้โปรแกรมทำอะไรเท่านั้นโดยไม่ต้องทราบว่าทำได้อย่างไร
- ทำให้เขียนโปรแกรมได้ง่ายและรวดเร็วกว่าภาษาระดับสูงในยุคที่ 3 ที่มีการเขียนโปรแกรมแบบบอกขั้นตอนการทำงาน (Procedural language)
- ภาษาระดับสูงมาก ทำงานเหมือนกับภาษาพูดว่าต้องการอะไรและเขียนเหมือนภาษาอังกฤษ

TABLE FILE SALES

SUM UNITS BY MONTH BY CUSTOMER BY PRODUCT

ON CUSTOMMER SUBTOTAL PAGE BREAK

END

ยุคที่ 4 : ภาษาระดับสูงมาก (Very high-level Language)

- ตัวอย่างภาษาคอมพิวเตอร์ในยุคที่ 4 ประกอบด้วย Report Generators, Query Language, Application Generators และ Interactive Database Management System Programs
- ภาษาที่ใช้สำหรับเรียกดูข้อมูลจากฐานข้อมูลได้ เรียกว่า ภาษาสอบถาม (Query languages) จัดเป็นภาษาในยุคที่ 4 ซึ่งสามารถใช้ค้นคืนสารสนเทศของฐานข้อมูล มาตรฐานของภาษาชนิดนี้ขึ้นอยู่กับฐานข้อมูลที่แตกต่างกัน ที่นิยมใช้กันมากที่สุดคือ SQL (Structured Query Language) และนอกจากนี้ยังมีภาษา Query By Example หรือ QBE ที่ได้รับความนิยมในการใช้งาน
- Report Generator หรือ Report Writer คือ โปรแกรมสำหรับผู้ใช้ (End user) ที่ใช้สำหรับสร้างรายงาน รายงานอาจแสดงที่เครื่องพิมพ์หรือจอภาพก็ได้อาจจะแสดงทั้งหมดหรือบางส่วนของฐานข้อมูลก็ได้ อาจจะกำหนดรูปแบบบรรทัดคอลัมน์ส่วนหัวรายงาน และอื่น ๆ ได้
- Application Generators คือ เครื่องมือของผู้เขียนโปรแกรมที่ใช้ในการสร้างโปรแกรมประยุกต์จากการอธิบายปัญหาได้เร็วกว่าการเขียนโปรแกรมทั่ว ๆ ไป

ยุคที่ 4 : ภาษาระดับสูงมาก (Very high-level Language)

- ข้อดีของภาษาคอมพิวเตอร์ในยุคที่ 4 ได้แก่
 - การเขียนโปรแกรมจะสั้นและง่าย เพราะเน้นที่ผลลัพธ์ของงานว่าต้องการอะไร โดยไม่สนใจว่าจะทำได้อย่างไร
 - การเขียนคำสั่ง สามารถทำได้ง่ายและแก้ไขเปลี่ยนแปลงโปรแกรมได้สะดวก ทำให้พัฒนาโปรแกรมได้รวดเร็วขึ้น
 - ผู้เขียนโปรแกรมสามารถเขียนโปรแกรมได้เร็ว โดยไม่ต้องเสียเวลาอบรม หรือมีความรู้ด้านการเขียนโปรแกรม เพราะชุดคำสั่งเหมือนภาษาพูด
 - ผู้เขียนโปรแกรมไม่จำเป็นต้องทราบถึงฮาร์ดแวร์ของเครื่องและโครงสร้างคำสั่งของภาษาโปรแกรม

ยุคที่ 5 : ภาษาธรรมชาติ(Natural Language)

- การเขียนคำสั่ง หรือสั่งงานคอมพิวเตอร์ทำงานโดยใช้ภาษาธรรมชาติต่าง ๆ เช่น ภาพ หรือ เสียง โดยไม่สนใจรูปแบบไวยากรณ์หรือโครงสร้างของภาษามากนัก ซึ่งคอมพิวเตอร์จะพยายามคิดวิเคราะห์ และแปลความหมายโดยอาศัยการเรียนรู้ด้วยตนเองและระบบองค์ความรู้ (Knowledge Base System) มาช่วยแปลความหมายของคำสั่งต่าง ๆ และตอบสนองต่อผู้ใช้งาน
- ตัวอย่างภาษาคอมพิวเตอร์ในยุคที่ 5 เช่น

SUM SHIPMENTS BY STATE BY DATE

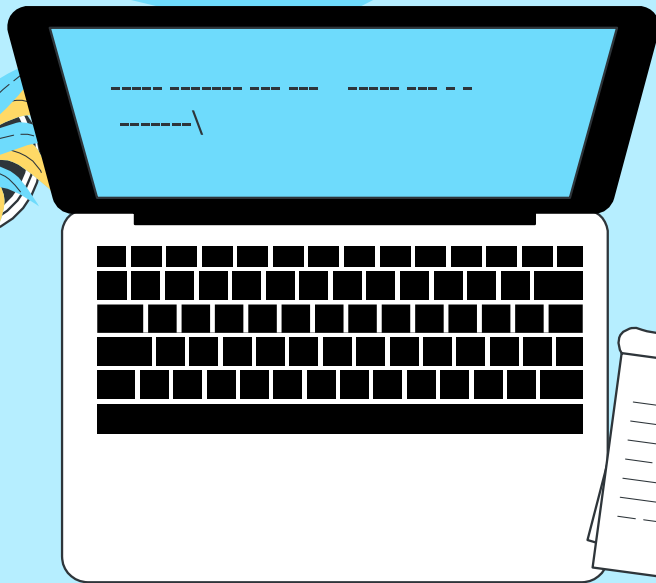
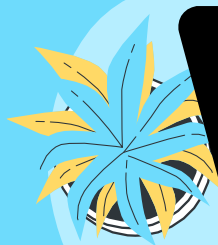
- ข้อดีของภาษาคอมพิวเตอร์ในยุคที่ 5 คือ ผู้เขียนโปรแกรมสามารถเขียนโปรแกรมได้เร็ว โดยไม่ต้องมีความรู้ด้านการเขียนโปรแกรม แต่คอมพิวเตอร์ที่ใช้โปรแกรมต้องมีระบบรับคำสั่ง และประมวลผลแบบอัจฉริยะสามารถตอบสนองและทำงานได้หลายแบบ

ภาษาโปรแกรมคอมพิวเตอร์ที่ใช้เขียนโปรแกรมเพื่อแก้ปัญหาต่าง ๆ มีมากมายหลายภาษา ซึ่งแต่ละภาษาโปรแกรมมีข้อดีและเหมาะกับใช้งานต่าง ๆ กัน ในการเลือกใช้ภาษาโปรแกรมคอมพิวเตอร์มาใช้งาน ควรพิจารณาสิ่งต่าง ๆ ดังนี้คือ

1. เลือกภาษาโปรแกรมคอมพิวเตอร์ที่เหมาะสมกับความชำนาญของบุคลากร หรือนักเขียนโปรแกรม
2. คำนึงถึงเวลาและค่าใช้จ่ายในการพัฒนาและเขียนโปรแกรม
3. เลือกภาษาโปรแกรมคอมพิวเตอร์ที่นักเขียนโปรแกรมเข้าใจง่าย มีการประยุกต์ใช้งานได้อย่างแพร่หลาย และสามารถพัฒนาต่อไปในอนาคตได้
4. เลือกภาษาโปรแกรมคอมพิวเตอร์ให้เหมาะกับฮาร์ดแวร์ และสามารถทำงานร่วมกันได้ระหว่างอุปกรณ์คอมพิวเตอร์

02

โครงสร้างทางตรรกะใน
การเขียนโปรแกรม



วิธีการแก้ปัญหาของคอมพิวเตอร์อย่างมีประสิทธิภาพ :

1. เขียนโปรแกรมโดยการใช้การแบ่งส่วนโปรแกรมแทน หรือที่เรียกว่า โมดูล (Modules)
2. ใช้ 4 โครงสร้างทางตรรกะ (logic structures) ช่วยในการเขียนโปรแกรม
 - a. Sequential structure
 - เป็นการประมวลผลคำสั่งตามลำดับขั้น ตั้งแต่คำสั่งแรกจนถึงคำสั่งสุดท้าย
 - b. Decision structure
 - แยกการประมวลผลคำสั่งเป็นกลุ่มๆ ในการตัดสินใจ
 - c. Loop structure
 - การประมวลผลคำสั่งที่เหมือนกันในกลุ่มเดียวกันซ้ำๆ กัน
 - d. Case structure
 - เป็นการประมวลผลกลุ่มคำสั่ง จากหลายๆ กลุ่ม
3. หลีกเลี่ยงการเขียนโปรแกรมที่เหมือน ๆ กัน โดยการใช้โมดูลแทน
4. ใช้เทคนิคต่าง ๆ ช่วยในการปรับปรุงโปรแกรม รวมถึงโครงสร้างของตรรกะทั้ง 4 ข้างต้น การตั้งชื่อที่เหมาะสมของตัวแปร มีคำอธิบายหรือหมายเหตุ (Comment) ในแต่ละบรรทัดในโปรแกรม และการเยื้องในแต่ละบรรทัดที่เหมาะสม

การใช้คำสั่งเงื่อนไข IF/THEN/ELSE

- การที่จะบอกให้คอมพิวเตอร์ทำงานตามโปรแกรมที่มีคำสั่งเงื่อนไข
- IF เงื่อนไขเป็นจริง, THEN ประมวลผลคำสั่งที่กำหนด ELSE ประมวลผลคำสั่งอื่นๆ

ในส่วน ELSE คือ ส่วนเสริม, จะมีหรือไม่มีคำสั่งก็ได้ ใช้ในกรณีที่เงื่อนไขเป็นเท็จ

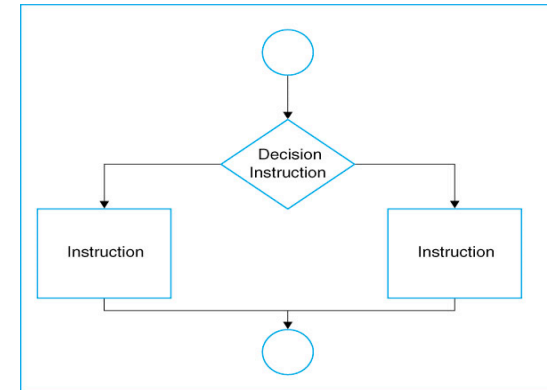
Algorithm:

IF <condition(s)> THEN

instruction(s) เครื่องจะทำงานในคำสั่งนี้เมื่อเงื่อนไขเป็นจริง (T)

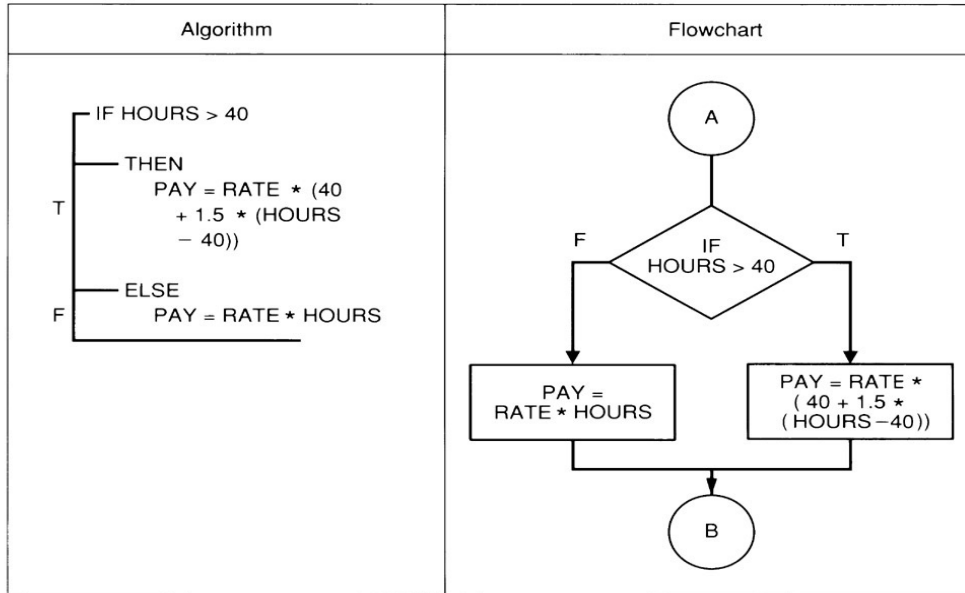
ELSE

instruction(s) เครื่องจะทำงานในคำสั่งนี้เมื่อเงื่อนไขเป็นเท็จ (F)



การใช้คำสั่งเงื่อนไข IF/THEN/ELSE

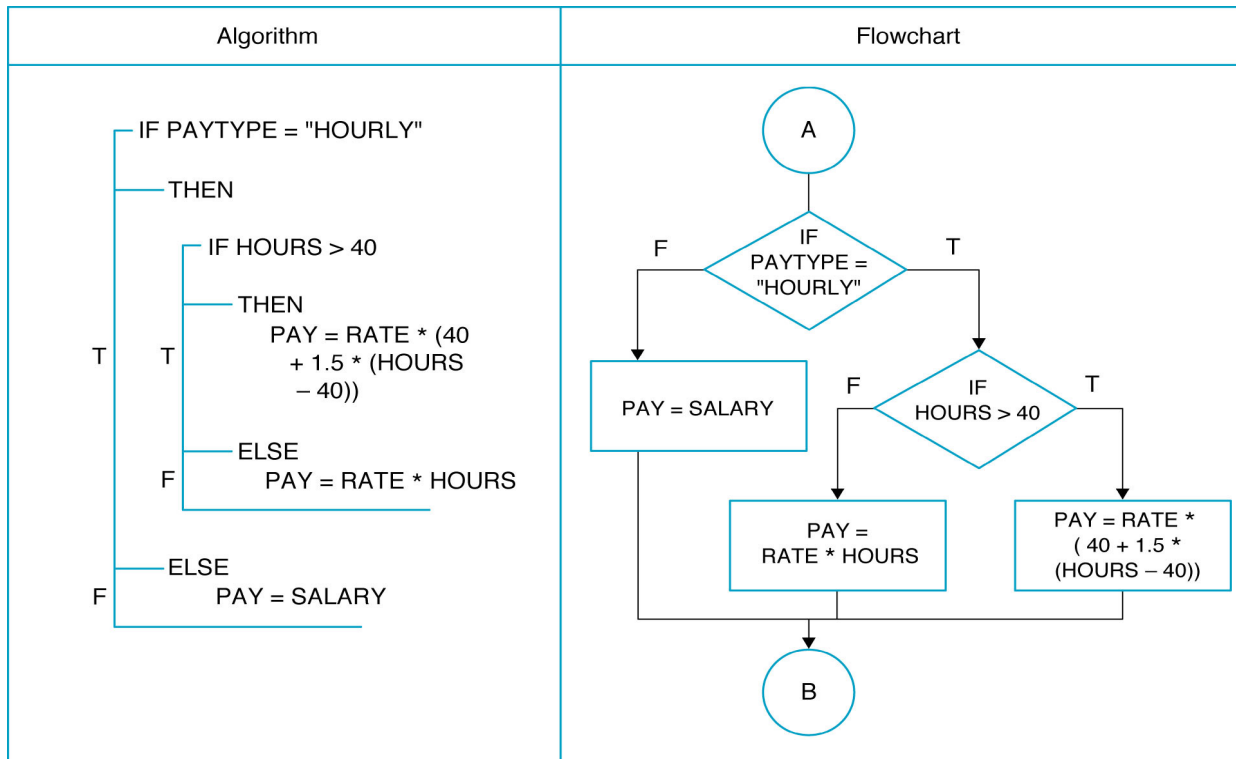
- จงคำนวณการจ่ายเงินค่าจ้างและค่าล่วงเวลา (กรณีเกิน 40 ชั่วโมง) จ่าย 1.5 เท่า
IF ชั่วโมงการทำงานมากกว่า 40, THEN คำนวณค่าล่วงเวลา, หรือ ELSE คำนวณค่าจ้างปกติ



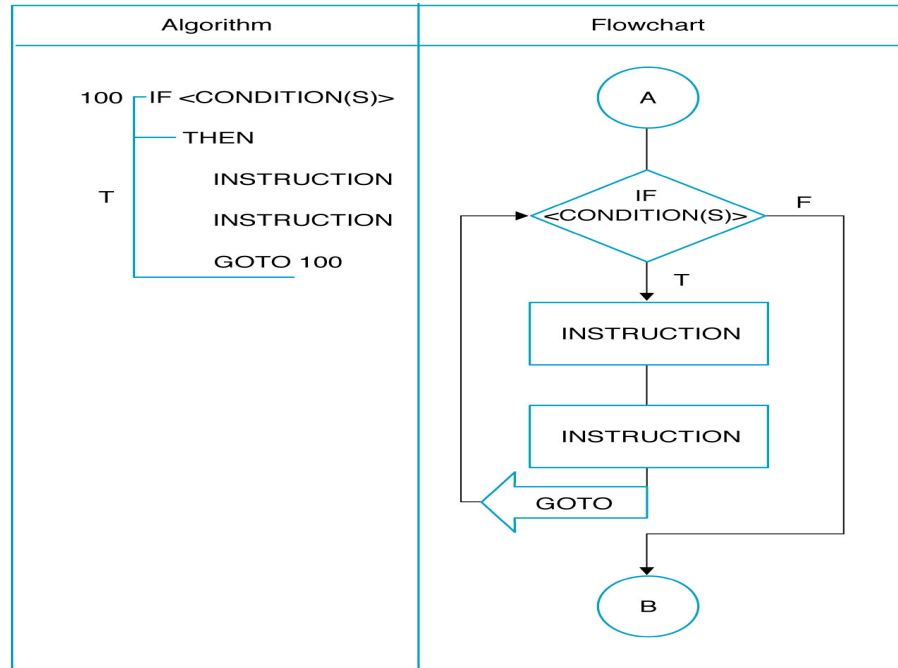
Note: For all flowcharts with decision blocks, T = TRUE and F = FALSE

การใช้คำสั่งเงื่อนไข NESTED IF/THEN/ELSE

- Multiple decisions เป็นการตัดสินใจหลายทางเลือก
- คำสั่งที่ต้องทำตามเป็นชุดคำสั่งในแต่ละระดับของการตัดสินใจ



การใช้คำสั่งเงื่อนไข WHILE loop



+ การใช้คำสั่งเงื่อนไข WHILE loop

Example: Get the sum of 1, 2, 3, ..., 100

Algorithm:

Set the number = 1

Set the total = 0

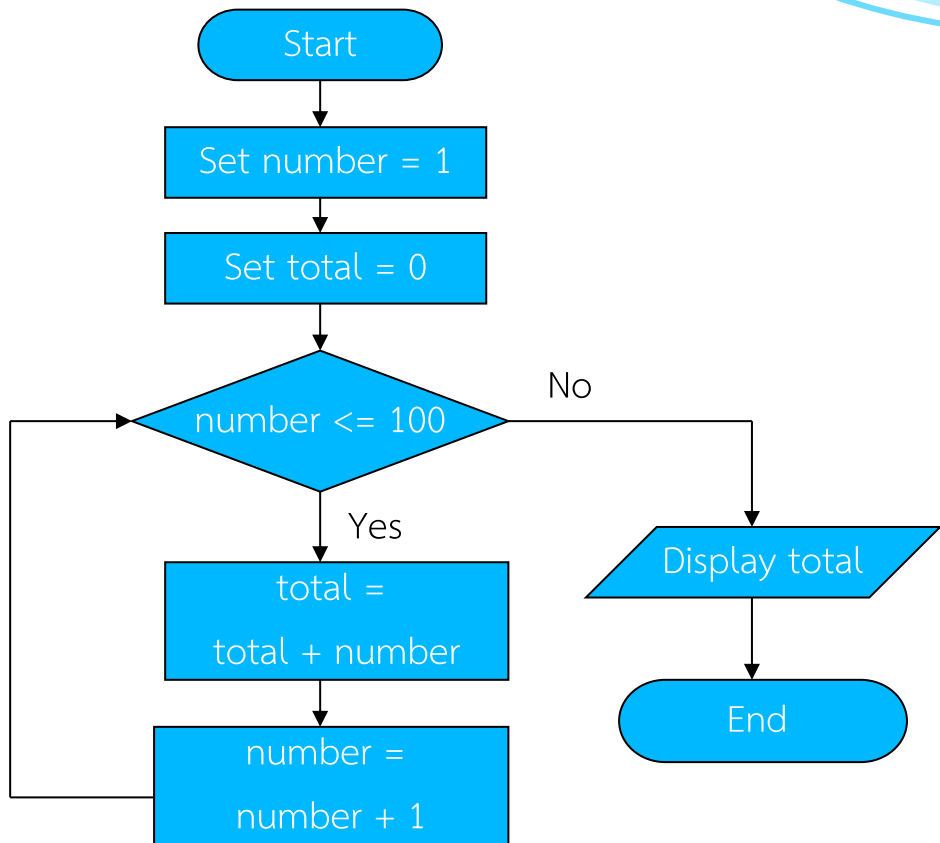
While (number <= 100)

total = total + number

number = number + 1

End While

Display total



+ การใช้คำสั่งเงื่อนไข DO...WHILE Loop

Example: Get the sum of 1, 2, 3, ..., 100

Algorithm:

Set the number = 1

Set the total = 0

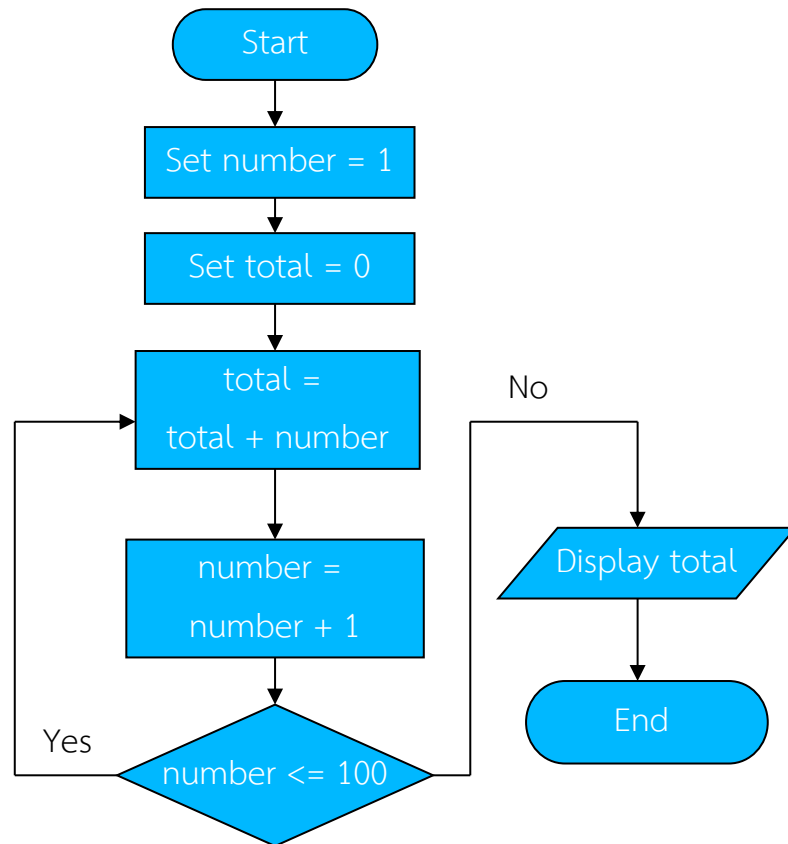
Do

total = total + number

number = number + 1

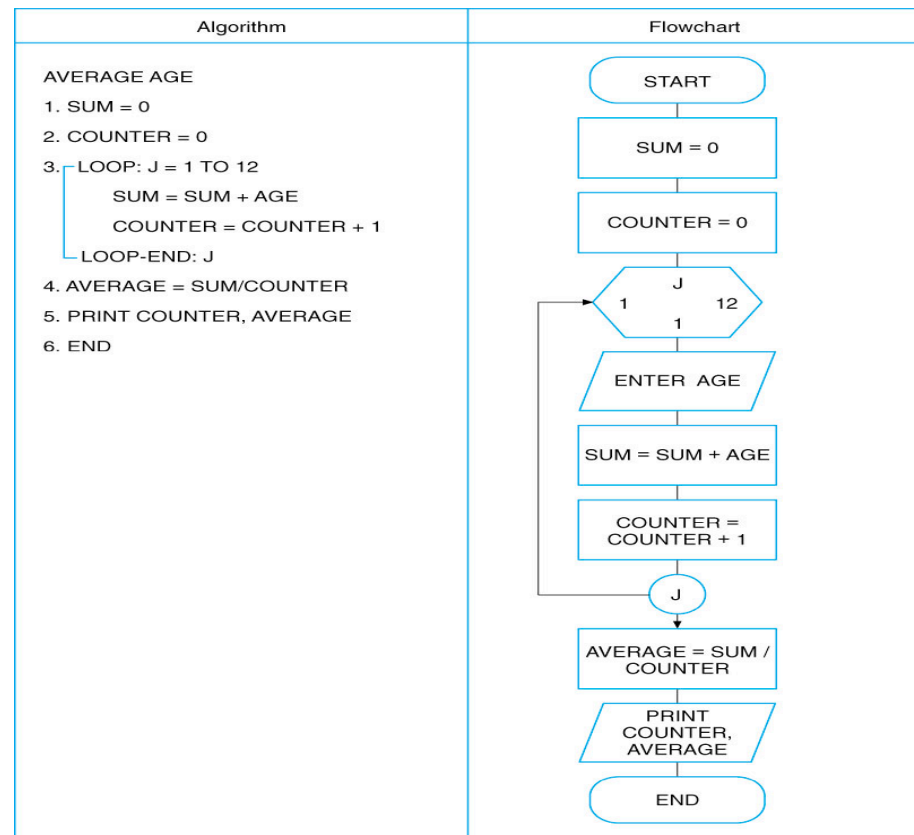
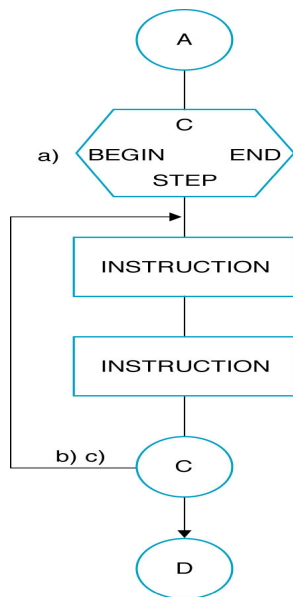
While (number <= 100)

Display total



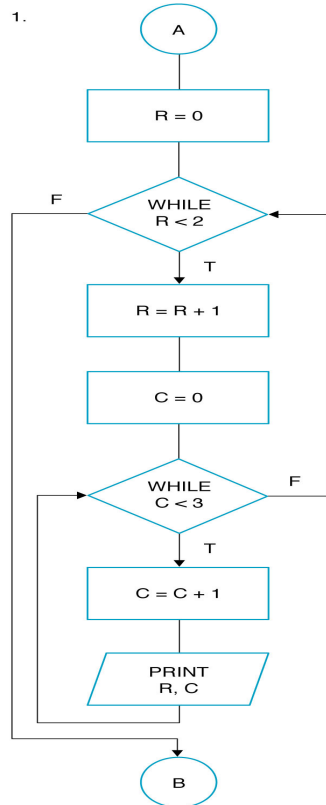
การใช้คำสั่งเงื่อนไข Automatic-Counter Loop

- ใช้ตัวแปรเป็นตัวนับที่เริ่มนับตามจำนวนที่ระบุ และเพิ่มตัวแปรทุกครั้งที่ประมวลผลลูป
- ค่าเริ่มต้น ค่าสิ้นสุด และค่าที่เพิ่มขึ้นอาจเป็นค่าคงที่ ไม่ควรเปลี่ยนแปลงระหว่างการประมวลผลคำสั่งในลูป

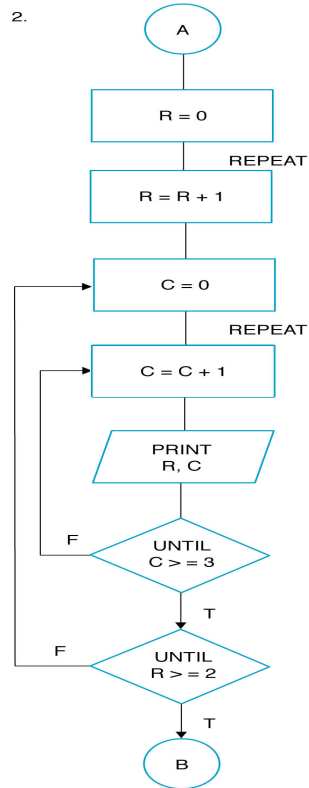


การใช้คำสั่งเงื่อนไข NESTED LOOP

1.



2.



```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    for( int i=1; i<=3; i++ ){
```

```
        for( int j=1; j<=3; j++ ){
```

```
            printf("loop แรก = %d / loop สอง = %d\n", i, j);
```

```
        }
```

```
    }
```

```
}
```

ผลลัพธ์

loop แรก = 1 / loop สอง = 1

loop แรก = 1 / loop สอง = 2

loop แรก = 1 / loop สอง = 3

loop แรก = 2 / loop สอง = 1

loop แรก = 2 / loop สอง = 2

loop แรก = 2 / loop สอง = 3

loop แรก = 3 / loop สอง = 1

loop แรก = 3 / loop สอง = 2

loop แรก = 3 / loop สอง = 3

การใช้คำสั่งเงื่อนไข Case Logic Structure

Algorithm:

CASE OF VARIABLE

= constant1:

actions for VARIABLE = constant1

= constants2:

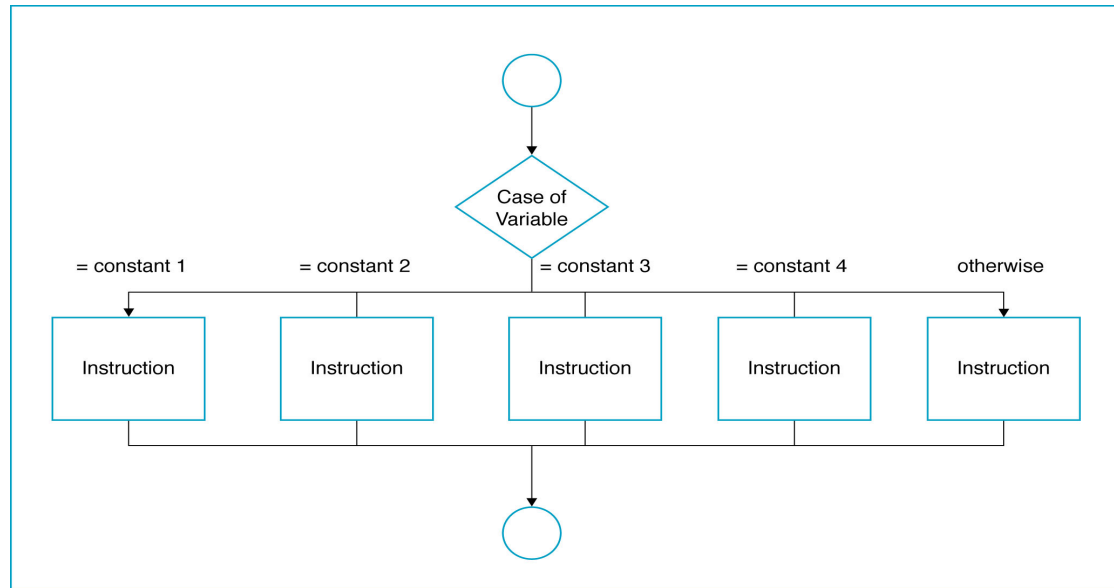
actions for VARIABLE = constant2

...

OTHERWISE:

Actions for VARIABLE = anything else

END-OF-CASE



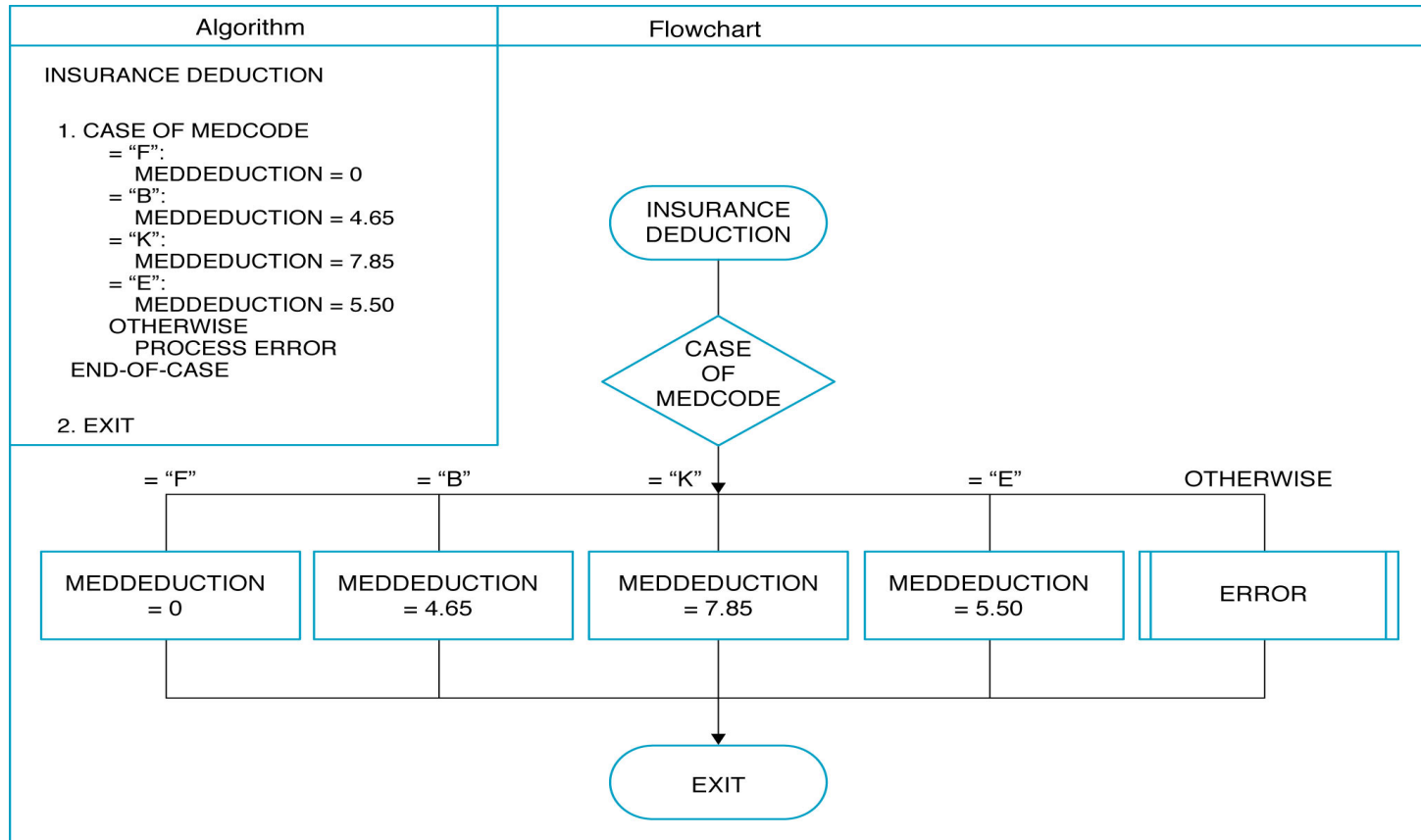
การใช้คำสั่งเงื่อนไข Case Logic Structure

Example: A company has four different medical plans. The programmer has given each plan a code corresponding to the beginning initial of the company:

Plan 1 = F, Plan 2 = B, Plan 3 = K, Plan 4 = E.

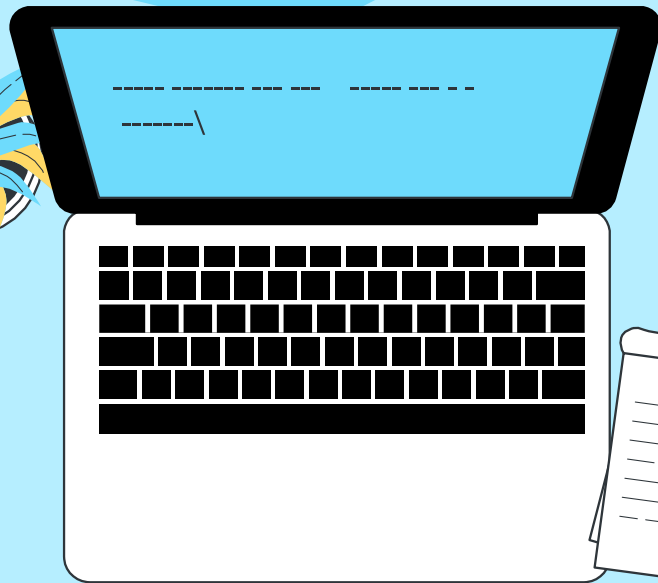
The company pays for all of Plan 1. The individual has to pay for part of the others. The payroll deduction for Plan 2 = 4.65, for Plan 3 = 7.85, and for Plan 4 = 5.50. Any other codes are considered in error. Write the algorithm and draw the flowchart for a module to determine the payroll deduction.

การใช้คำสั่งเงื่อนไข Case Logic Structure



03

ข้อมูล ตัวแปร ฟังก์ชัน
และตัวดำเนินการ



Data - Constants vs. Variables

Constants (ค่าคงที่)

- ข้อมูลชนิดใดก็ได้ เช่น ตัวเลข, ตัวหนังสือ ที่มีค่าคงที่
- ไม่สามารถเปลี่ยนแปลงค่าได้ตลอดทั้งโปรแกรม
- บางภาษาสามารถกำหนดชื่อให้ค่าคงที่ได้
- ค่าคงที่จะถูกเก็บไว้ที่ตำแหน่งใดตำแหน่งหนึ่งใน memory โดยใช้ชื่อในการอ้างถึงตำแหน่งนั้น
- เช่น กำหนดให้ PI แทนเลข 3.14159

Data - Constants vs. Variables [cont.]

Variables (ตัวแปร)

- ตรงข้ามกับ constants
- ค่าของตัวแปรสามารถเปลี่ยนแปลงได้ตลอดทั้งโปรแกรม (ถ้ามีคำสั่งใดไปสั่งให้เปลี่ยนแปลงค่า)
- ข้อมูลชนิดใดก็ได้ เช่น ตัวเลข, ตัวหนังสือ ที่มีค่าคงที่
- ต้องมีการกำหนดชื่อให้ตัวแปรเสมอ - เป็นการอ้างถึงตำแหน่งในหน่วยความจำซึ่งเก็บข้อมูลของตัวแปรนี้อยู่
- เมื่อข้อมูลในตัวแปรเปลี่ยน ข้อมูลเก่าจะต้องหายไป

Data - Constants vs. Variables [cont.]

กฎพื้นฐานของการตั้งชื่อตัวแปร

- สื่อความหมาย เช่น hours, payRate, StdName
- ไม่มีช่องว่าง!!!
- ขึ้นต้นด้วยตัวอักษร
- ไม่ใช่สัญลักษณ์พิเศษ
- ใช้ชื่อเดิมตลอดทั้งโปรแกรม
- บางภาษา ตัวพิมพ์ใหญ่/เล็ก ถือเป็นคนละชื่อกัน
- ค่าคงที่ - เหมือนตัวแปรแต่นิยมตั้งใช้ตัวพิมพ์ใหญ่ทั้งหมด

Data - Constants vs. Variables [cont.]

Ex. Variable names

- Date
- sum
- maximum
- temperature
- StudentName (ภาษา C)
- yearOfBirth (ภาษา JAVA)

Ex. Constants names

- PI
- MULTIPLIER
- MAXIMUM

Data – Data Types [cont.]

- คอมพิวเตอร์ไม่สามารถเข้าใจชนิดข้อมูลได้ด้วยตัวเอง
- โปรแกรมเมอร์ต้องกำหนดให้ (ทั้ง var. และ const.) ตามความเหมาะสม
- แบ่งได้เป็น 6 ชนิดหลัก
 - Numeric : Integer
 - Numeric : Real
 - Character
 - String
 - Logical
 - Other data types ขึ้นอยู่กับแต่ละภาษาโปรแกรม

Data Type	Data Set	Examples
Numeric: Integer	All whole numbers	3580 -46
Numeric: Real	All real numbers (whole + decimal)	-3792.91 4739416.0 0.00246
Character (surrounded by quotation marks)	All letters, numbers, and special symbols	"A" "a" "M" "z" "k" "1" "5" "7" "8" "0" "+", "=", "(", "%", "\$"
String (surrounded by quotation marks)	Combinations of more than one character	"Arcata" "95521" "707-444-5555"
Logical	True False	True False

Process - Functions

- กลุ่มของคำสั่งขนาดเล็กที่ทำงานเฉพาะบางงาน
- มีการคืนค่า (return) ผลลัพธ์กลับมา
- ช่วยให้เขียนโปรแกรมง่ายขึ้น, ไวขึ้น
- มีในภาษาโปรแกรมต่างๆ (มาก/น้อยขึ้นอยู่กับแต่ละภาษา)
- สามารถสร้างเพิ่มเติมเองได้ตามต้องการ
- ตัวอย่างฟังก์ชัน(กลุ่ม) เช่น Mathematical, String, Conversion, Statistical, Time, ฯลฯ

Process – Functions [cont.]

Group	Example	Result
Mathematical	Sqrt(4)	2
	Abs(-3)	3
	Round(3.7259,2)	3.73
	Integer(5.352)	5

Process – Functions [cont.]

Group	Example	Result
String	Left(“dang”,3)	“dan”
	Length(“phayao”)	6
	Right(“dang”,2)	“ng”
Conversion	Value(“5.37”)	5.37
	String(5.37)	“5.37”

Process – Functions [cont.]

Group	Example	Result
Statistical	Average(5,3,8,6)	5.5
	Max(5,3,8,6)	8
	Min(5,3,8,6)	3
	Sum(5,3,8,6)	22
Time	Date()	27/7/55
	Time()	11:05:55

Process – Operators (ตัวดำเนินการ)

- สัญลักษณ์เพื่อใช้บอกคอมพิวเตอร์ให้ประมวลผลข้อมูล
- process ข้อมูล (Operand) ตัวไหน? อย่างไร?
- แต่ละภาษาอาจใช้สัญลักษณ์ที่แตกต่างกันได้ เช่น “ไม่เท่ากับ” `<>` (Visual Basic) `!=` (C)
- ประกอบไปด้วย 3 กลุ่มมาตรฐาน คือ Mathematical, Relational, Logical
- บางภาษาอาจมีกลุ่มอื่น ๆ เพิ่มเติมได้ เช่น ภาษา C มีกลุ่ม Bitwise Operator

Process – Operators (ตัวดำเนินการ)

อ้างอิงภาษาซี

Group	Symbol	Example	Result
Mathematical	+	$3.0 + 5.2$	8.2
	-	$4 - 9$	-5
	*	$5 * 4$	20
	/	$9.0 / 4.0$	2.25
	% (MOD)	$9 \% 4$	1
	^ (not in C)	$3 ^ 2$	9

Process – Operators (ตัวดำเนินการ)

อ้างอิงภาษาซี

Group	Symbol	Example	Result
Relational	==	5 == 7	<i>False</i>
	!=	'A' != 'a'	<i>True</i>
	<	8 < 7	<i>False</i>
	<=	5 <= 5	<i>True</i>
	>	10 > 1	<i>True</i>
	>=	1 >= 10	<i>False</i>

Process – Operators (ตัวดำเนินการ)

อ้างอิงภาษาซี

Group	Symbol	Example	Result
Logical	&&	<i>True</i> && <i>True</i>	<i>True</i>
		<i>False</i> <i>False</i>	<i>False</i>
	!	! <i>True</i>	<i>False</i>

Process – Operators (ตัวดำเนินการ)

A	B	A && B	A B	!A
<i>True</i>	<i>True</i>	<i>True</i>	<i>True</i>	<i>False</i>
<i>True</i>	<i>False</i>	<i>False</i>	<i>True</i>	<i>False</i>
<i>False</i>	<i>True</i>	<i>False</i>	<i>True</i>	<i>True</i>
<i>False</i>	<i>False</i>	<i>False</i>	<i>False</i>	<i>True</i>

Process – Operators (ตัวดำเนินการ)

ลำดับความสำคัญของ operator

1. () โดยในวงเล็บให้ทำงานตามลำดับความสำคัญ
2. !
3. * , / , %
4. + , -
5. < , <= , > , >=
6. == , !=
7. &&
8. ||

ถ้ามีความสำคัญเท่ากันให้ทำจากซ้ายไปขวา

Exercise 1

ในโปรแกรมต้องมีการทดสอบค่าเพื่อตัดสินใจทำงาน โดยเงื่อนไขที่ตรวจสอบคือ X น้อยกว่า $Y + 5$ หรือไม่ จงเขียนเงื่อนไขนี้ในรูปของ expression

Exercise 2

ถ้าในโปรแกรมมีคำสั่งกำหนดค่า (assignment statement) ดังต่อไปนี้

$$C = 5 * (X + Y) - 4 * Y / (Z + 6)$$

และกำหนดให้ตัวแปรต่างๆ มีค่าดังนี้

$$X = 2, Y = 3, Z = 6$$

ตัวแปร C จะมีค่าเท่าใด

Exercise 3

ถ้าในโปรแกรมมีคำสั่งกำหนดค่า (assignment statement) ดังต่อไปนี้

$$C = A - 2 > B$$

และกำหนดให้ตัวแปรต่างๆ มีค่าดังนี้

$$A = 6, B = 8$$

ตัวแปร C จะมีค่าเท่าใด

Exercise 4

ถ้าในโปรแกรมมีคำสั่งกำหนดค่า (assignment statement) ดังต่อไปนี้

$$C = A \ \&\& \ B \ || \ C \ \&\& \ A$$

และกำหนดให้ตัวแปรต่างๆ มีค่าดังนี้

$A = \text{True}$, $B = \text{False}$, $C = \text{True}$

ตัวแปร C จะมีค่าเท่าใด

Exercise 7

ร้านค้าแห่งหนึ่งซึ่งมีระบบสมาชิก ได้จัดโปรโมชั่นลดราคาสินค้า โดยมีเงื่อนไข คือ ต้องเป็นสมาชิก และมียอดซื้อระหว่าง 500 ถึง 5,000 บาท จงเขียน expression เพื่อตรวจสอบว่าลูกค้าที่มาซื้อของจะได้ส่วนลดหรือไม่ (ต้องสร้างตัวแปรอะไรบ้าง??)

แนวคิด

ต้องมีตัวแปรเพื่อเก็บข้อมูลอะไรบ้าง, ตัวแปรชนิดใด และจะเขียน expression อย่างไร

Thanks!

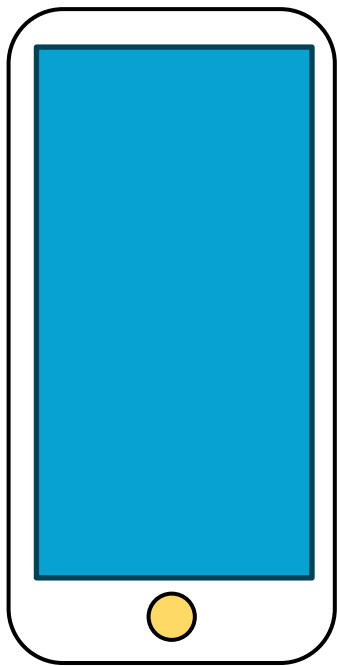


แบบฝึกหัดท้ายบท

จงเขียน Algorithm และ Flowchart

1. หอพักแห่งหนึ่งมีราคาห้องพักแต่ละแบบดังนี้
 - ห้องแบบพัก 2 คน : ฿ 1850
 - ห้องแบบพัก 3 คน : ฿ 2450
 - ห้องแบบพัก 4 คน : ฿ 3250
 - นอกเหนือจากนี้ ให้พิมพ์บอกว่า “ไม่มี”
 - ถ้าผู้พักจองก่อน 15 วัน ลด 20%
 - ถ้าผู้พักจองก่อนวันเข้าพัก 1-14 วัน ลด 10%
 - นอกเหนือจากเงื่อนไขข้างต้นไม่ลดราคา
 - จ่ายค่าห้องล่วงหน้า 2 เดือน
 - และพิมพ์ผลลัพธ์ราคาห้องพักออกมา

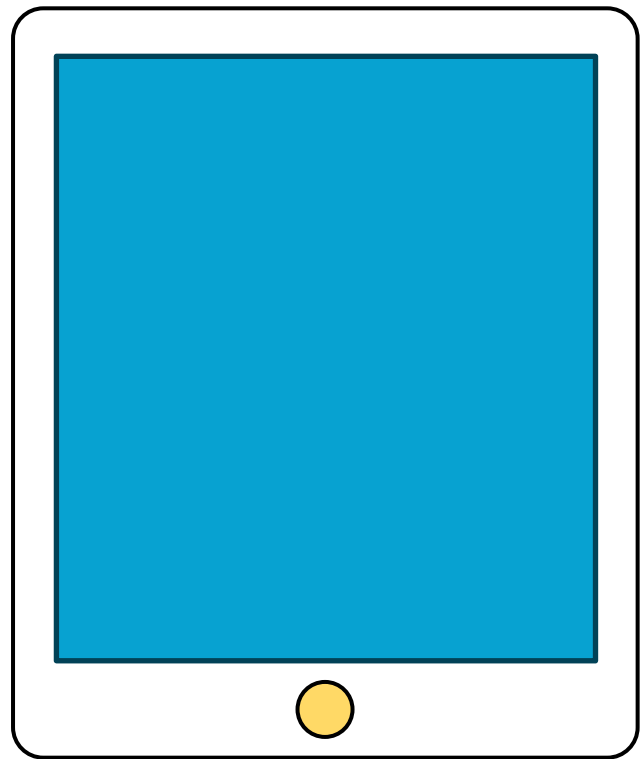


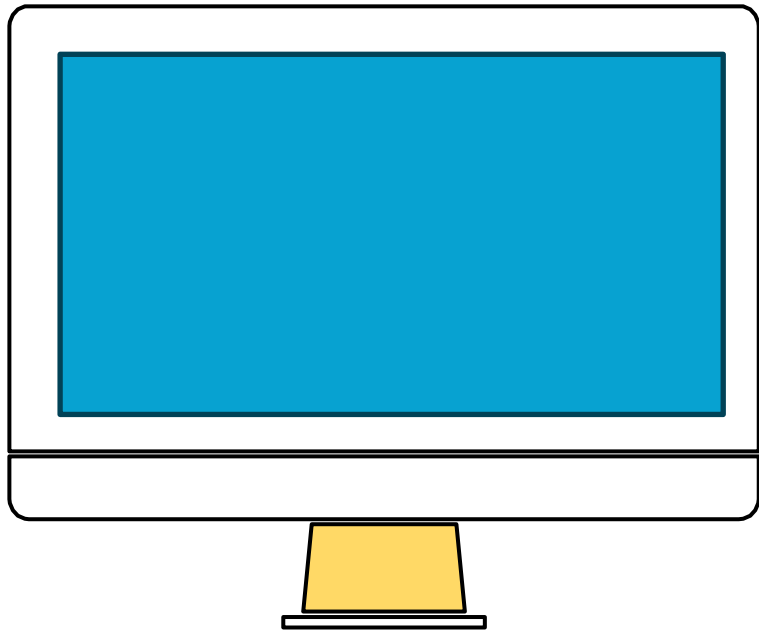


2. ให้นิสิตกำหนดชื่อตัวแปรและชนิดข้อมูลดังต่อไปนี้

- จำนวนนิสิตในห้องเรียน
- อุณหภูมิ
- ปริมาณรถยนต์ที่เข้า/ออกมหาวิทยาลัย
- ราคาโทรศัพท์มือถือ
- วันที่
- ชื่อนิสิต
- หมายเลขบัตรประชาชน

3. ในหลาย ๆ ครั้ง โปรแกรมเมอร์ต้องมีการตรวจสอบผลลัพธ์ที่เป็นไปได้ทั้งหมด เช่น ในการตรวจสอบเงื่อนไขของโปรแกรม (ซึ่งผลลัพธ์เป็นค่า logical) ดังนั้นให้นิสิตเขียนเงื่อนไขทั้งหมดที่เป็นไปได้ของ expression (เขียนตารางค่าความจริง (Truth Table) ดังต่อไปนี้

$$(A \ \&\& \ B) \ || \ (A \ \&\& \ C)$$




ถ้าในโปรแกรมมีคำสั่งกำหนดค่า (assignment statement)

ดังต่อไปนี้

$$F = !(A < B) \&\& (C \parallel D)$$

และกำหนดให้ตัวแปรต่างๆ มีค่าดังนี้

$A = 4$, $B = 2$, $C = \text{True}$, $D = \text{False}$

ตัวแปร F จะมีค่าเท่าใด